Conv_Simple — Ladder-First Edition

Every program drawn as ladder rungs. No Structured Text until you ask for it.

Audience: First-time PLC programmer who wants to learn ladder before structured text | **PLC:** Allen-Bradley Micro 820 (2080-LC20-20QBB)
Drive: AutomationDirect GS10 DURApulse, Modbus RTU slave node 1
Project: `MIRA_v5_LD` (new project, doesn't touch deployed `MIRA_v5`)
IDE: Connected Components Workbench (CCW) v22+
Companion doc: `Conv_Simple_Complete.pdf` — the ST version of the same logic (read after you outgrow ladder)
Reference program: `plc/Micro820_v4.1.8_Program.st` (deployed, in ST — this doc is a parallel teaching mirror, not a replacement)
You will understand after reading: how to draw the entire conveyor program — e-stop supervision, I/O decoding, state machine, Modbus polling, motor-control UDFB — as ladder rungs. Where ladder is clear, and where it gets painful enough that ST starts to look attractive.

> **Why a ladder-first edition exists at all**
> Maintenance technicians who came up before 2010 read ladder fluently and find ST alien. Half the panels in the field are still pure ladder. Even if you eventually adopt ST for state machines, you'll spend the rest of your career reading and modifying other people's ladder logic. Learning ladder first means: every contact, every coil, every timer, every MSG block has an unambiguous visual shape that you can point at on a printout and discuss with an electrician. ST has the same logic compressed into text — faster to write once you're fluent, slower to teach and slower to share with a tech who doesn't program.

📐 Ladder primitives you'll use in this doc

Symbol	Name	What it does
`─[ ]─`	XIC (eXamine If Closed)	NO contact. Passes power when bit is TRUE.
`─[/]─`	XIO (eXamine If Open)	NC contact. Passes power when bit is FALSE.
`─( )─`	OTE (Output Energize)	Coil. Bit follows rung state.
`─(S)─`	OTL (Latch / Set)	Sets bit TRUE when rung true. Stays set after.
`─(R)─`	OTU (Unlatch / Reset)	Sets bit FALSE when rung true. Stays clear after.
`[ONS]`	One-Shot	Passes power for exactly one scan on rising edge.
`[EQU A,B]`	Equal	Compare contact. Passes power when A = B.
`[GRT A,B]`	Greater Than	Compare contact. Passes power when A > B.
`[MOV S,D]`	Move	Output instruction. Copies source value into dest.
`[ADD A,B,D]`	Add	D := A + B. Output instruction.
`[COP S,D,N]`	Copy block	Copies N elements from S array to D array.
`[TON t,PT]`	Timer-On-Delay	Output FB. `.Q` goes TRUE after PT elapsed with IN high.
`[CTU c,PV]`	Count Up	Output FB. `.CV` increments on rising edge of CU.
`[MSG_MODBUS m]`	Modbus messenger	Output FB. Sends one Modbus packet when IN rises.

CCW exposes all of these in the Toolbox of any LD editor. Drag → drop → wire variable names.

ⓘ One-page cheat sheet — values and tags used throughout

Bench wiring + serial parameters

What	Value
PLC serial channel (embedded RS-485)	`2` **(NOT 0)**
Drive RJ45 pin 5 = SG+, pin 4 = SG-, pin 3 = SGND	—
Baud / format	`9600 8N2 RTU` (P09.01=96, P09.04=13)
Drive Modbus node	`1` (P09.00)

GS10 registers

FC03 monitor block	`0x2103` (8451), ElementCnt = 4
FC03 status word	`0x2101` (8449), ElementCnt = 1
FC06 Control Command	`0x2000` (8192)
FC06 Frequency Setpoint	`0x2001` (8193), value = Hz × 10
FC06 Fault Reset	`0x2002` (8194), data = `2`

Command word values (bench-verify before relying)

STOP	`1`
FWD + RUN	`18`
REV + RUN	`20`

Physical I/O map (Micro 820 embedded)

I/O	Tag	Use
`_IO_EM_DI_00`	SelectorFWD	NO contact, closed when knob LEFT
`_IO_EM_DI_01`	SelectorREV	NO contact, closed when knob RIGHT
`_IO_EM_DI_02`	EStopNC	NC contact (healthy = 1, pressed = 0)
`_IO_EM_DI_03`	EStopNO	NO contact (healthy = 0, pressed = 1)
`_IO_EM_DI_04`	PBRun	Illuminated green pushbutton, NO momentary
`_IO_EM_DI_05`	Sensor_1	Entry photoeye
`_IO_EM_DI_06`	Sensor_2	Exit photoeye
`_IO_EM_DO_00`	LightGreen	Running pilot light
`_IO_EM_DO_01`	LightRed	Fault / e-stop pilot light
`_IO_EM_DO_02`	ContactorQ1	Safety contactor coil (drops 3-phase power)

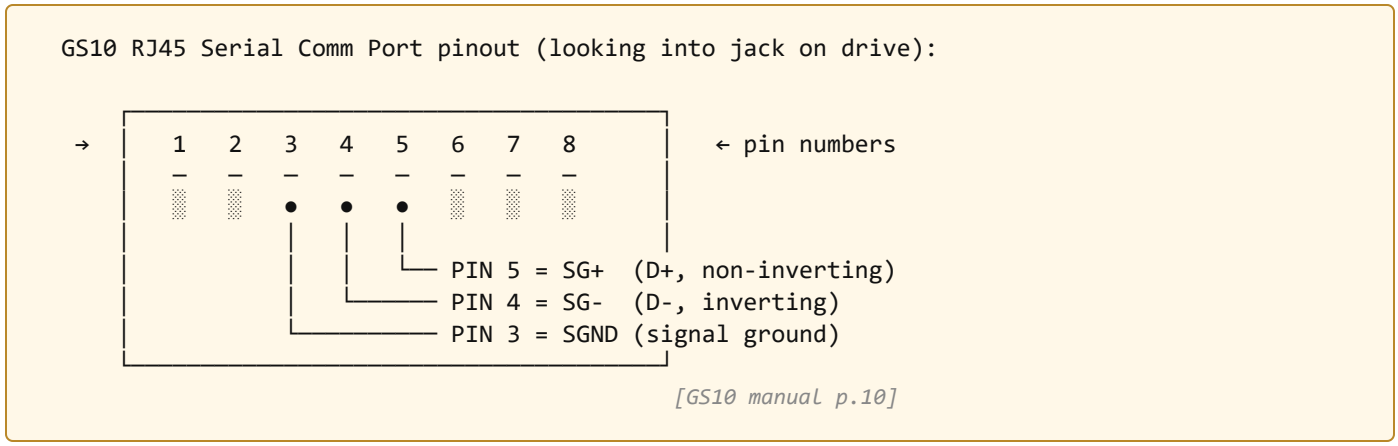
I/O	Tag	Use
`_IO_EM_DO_03`	PBRunLED	Green pushbutton's internal lamp

📖 Reading map (6 parts)

1. **Part 1 — Setup + prerequisites.** Wiring, drive keypad, fresh CCW project. Sections 1–3.
2. **Part 2 — Prog2-Safety in ladder.** E-stop XOR, I/O decoding, edge detection, contactor, state machine, LEDs. Sections 4–10.
3. **Part 3 — Prog3-Modbus in ladder.** Config loading via MOV, COP write buffers, polling timer, 4 MSG rungs. Sections 11–15.
4. **Part 4 — Prog4-Motor: build motor_starter as an LD UDFB.** Same pattern as the ST version, but every rung drawn. Sections 16–18.
5. **Part 5 — Test + troubleshoot.** Sections 19–20.
6. **Part 6 — When ladder hurts and ST starts to look attractive.** Honest map of which sections compress best. Sections 21–22.

PART 1 — Setup + prerequisites

1 RS-485 wiring (same prerequisite as the ST edition)



GS10 pin order is REVERSED from Allen-Bradley PowerFlex

Pin 5 = D+, Pin 4 = D-. AB convention is the opposite — follow AB by mistake and you get silence on the wire (no error, just no traffic). Verify with multimeter: 2.0–2.5V DC idle bias measured pin 5 (+) to pin 4 (-), drive powered, PLC offline.

2 GS10 keypad parameters (set once, save, power-cycle)

Param	Value	Meaning
P00.20	`3`	Frequency source = RS-485 comms
P00.21	`3`	Run command source = RS-485 comms
P09.00	`1`	Modbus slave address
P09.01	`96`	9600 baud
P09.04	`13`	RTU 8N2 format
P09.07	`2`	Comm-loss = stop the drive on timeout

3 Fresh CCW project for the ladder edition

This doc builds a parallel project so you don't disturb the deployed `MIRA_v5` ST project. New tags, new program names, new UDFB — everything ends in `_LD` or `_ladder` to make it obvious which world you're in.

1. ☐ CCW → File → New Project → name `MIRA_v5_LD`.
2. ☐ Controller type: **2080-LC20-20QBB**.

3. ☐ **Embedded Serial Port** → **Properties**: Protocol = Modbus RTU, Mode = Master, Baud = 9600, Parity = None, Stop Bits = 2, Handshake = None.
4. ☐ Project Organizer → Programs → right-click → **Add** → **New LD: Ladder Diagram**. Create three:
    - `Prog2-Safety` — e-stop, I/O decode, state machine, LEDs
    - `Prog3-Modbus-LD` — config loading, polling timer, 4 MSG rungs
    - `Prog4-Motor-LD` — UDFB instance call
5. ☐ Execution order: Prog2-Safety first, Prog3-Modbus-LD second, Prog4-Motor-LD third. Set in Programs → Properties → Order.
6. ☐ Global Variables → declare every tag listed in the cheatsheet plus the structs in Section 11. (Or import via CSV — see the deployed project's `VARIABLES_IMPORT.csv` for shape.)

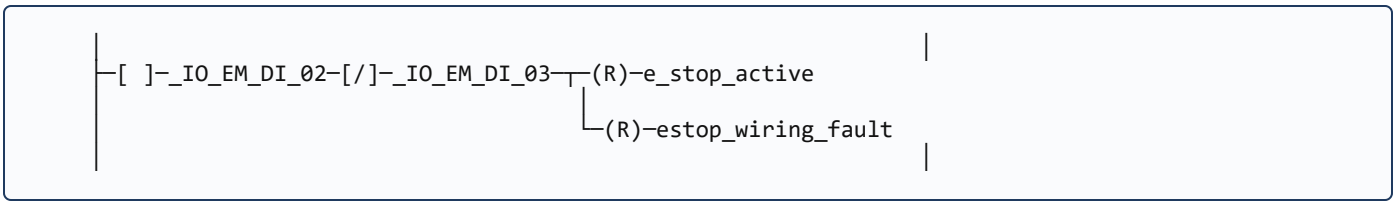
Why this matters: Keeping the LD project separate from the deployed ST project means you can fail freely. Download the LD project to the bench PLC, learn, break it, reload the deployed ST project anytime you want production back. No risk to existing work.

PART 2 — Prog2-Safety (all in ladder)

4 E-stop XOR supervision — 4 rungs

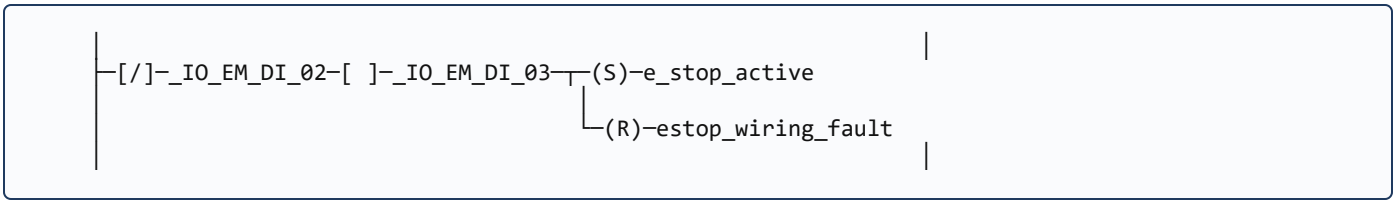
The dual-channel e-stop has one NC contact (`_IO_EM_DI_02`) and one NO contact (`_IO_EM_DI_03`). In a healthy installation they're always complementary: NC closed + NO open (released), or NC open + NO closed (pressed). If they're ever the same state, a wire broke or a contact welded — that's a wiring fault, latch the alarm and refuse to run until manually cleared.

Rung 4-1 — Released (NC closed + NO open)

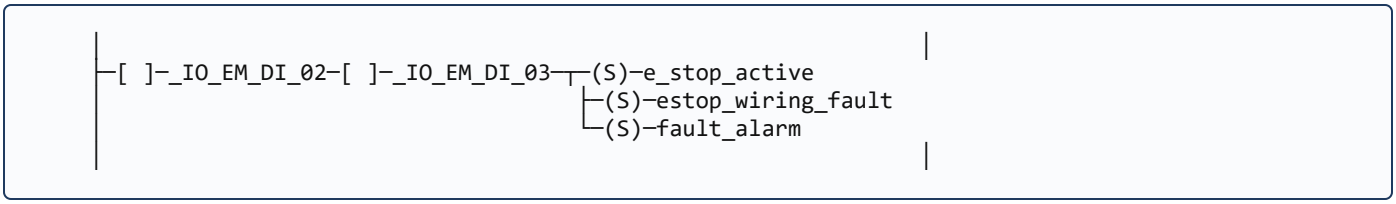


Both branches of the rung output. Two coils to the right of one composite contact group.

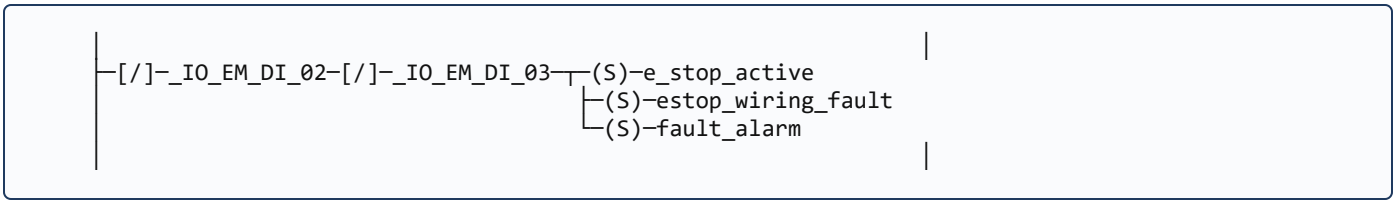
Rung 4-2 — Pressed (NC open + NO closed)



Rung 4-3 — Wiring fault (both closed)



Rung 4-4 — Wiring fault (both open)



Why this matters: Four rungs cover all four quadrants of the XOR truth table. The deployed ST version uses one `IF (_IO_EM_DI_02 XOR _IO_EM_DI_03)` statement — same logic, three lines instead of four rungs. The ladder version is more visual and easier for an electrician to verify with a multimeter, but four rungs is the price.

Rung 4-5 — Compute e_stop_ok (used later by the UDFB)

```
|                                                              |
├[/]─e_stop_active─[/]─estop_wiring_fault─( )─e_stop_ok
|                                                              |
```

5 Direction selector decoding — 4 rungs

The 3-position selector has two NO contacts wired to `_IO_EM_DI_00` (FWD) and `_IO_EM_DI_01` (REV). Center = both open. Left = FWD closed only. Right = REV closed only. Both-closed is a wiring fault (selector failed).

Rung 5-1 — dir_fwd

```
|                                                              |
├[ ]─_IO_EM_DI_00─[/]─_IO_EM_DI_01─( )─dir_fwd
|                                                              |
```

Rung 5-2 — dir_rev

```
|                                                              |
├[/]─_IO_EM_DI_00─[ ]─_IO_EM_DI_01─( )─dir_rev
|                                                              |
```

Rung 5-3 — dir_off

```
|                                                              |
├[/]─_IO_EM_DI_00─[/]─_IO_EM_DI_01─( )─dir_off
|                                                              |
```

Rung 5-4 — dir_fault

```
|                                                              |
├[ ]─_IO_EM_DI_00─[ ]─_IO_EM_DI_01─( )─dir_fault
|                                                              |
```

6 Rising-edge on the green pushbutton — 2 rungs

You need a one-scan "the button just got pressed" bit so a stuck-PB doesn't auto-restart the motor every scan. Ladder has two ways: an explicit ONS instruction, or a manual prev-value pattern. CCW supports both; ONS is more idiomatic.

Rung 6-1 — Using ONS (preferred)

```
|                                                              |
├[ ]─_IO_EM_DI_04─[ONS pb_ons_storage]─( )─button_rising
```

```
    |                                                                  |
```

ONS needs its own storage bit (`pb_ons_storage : BOOL`) — declare it as a global. The instruction handles the prev-value tracking internally.

Rung 6-1 alternate — Manual prev pattern (works if ONS isn't available)

```
    |                                                                  |
    ├─[ ]─_IO_EM_DI_04─[/]─prev_button─( )─button_rising
    |
    ├─[ ]─_IO_EM_DI_04─( )─prev_button
    |                                                                  |
```

*Two rungs. First produces the edge bit. Second shifts current value into prev for next scan. **Order matters** — edge rung must execute before the prev-update rung.*

Why this matters: Without rising-edge detection, holding the green PB while the latch is FALSE would re-fire the start every scan as long as the PB stays pressed. With rising-edge, the start only happens once per physical button press.

7 Safety contactor output — 1 rung

The hard-wired safety contactor (Q1, controlled by `_IO_EM_DO_02`) physically cuts 3-phase power to the VFD when de-energized. Energize it only when e-stop healthy and no wiring fault.

```
    |                                                                  |
    ├─[/]─e_stop_active─[/]─estop_wiring_fault─( )─_IO_EM_DO_02
    |                                                                  |
```

Contactor logic must be hardwired AND PLC-controlled
This rung is the PLC's view. The actual safety circuit ALSO must route the e-stop's hard contacts in series with the contactor coil so the PLC failing (powered off, frozen scan, downloaded with wrong logic) cannot leave the contactor energized while the e-stop is pressed. The PLC rung is a redundant guard, not the primary safety chain. See NFPA 79 §9.2.5 for the architectural requirement.

8 State machine in ladder — 5 states, ~20 transition rungs

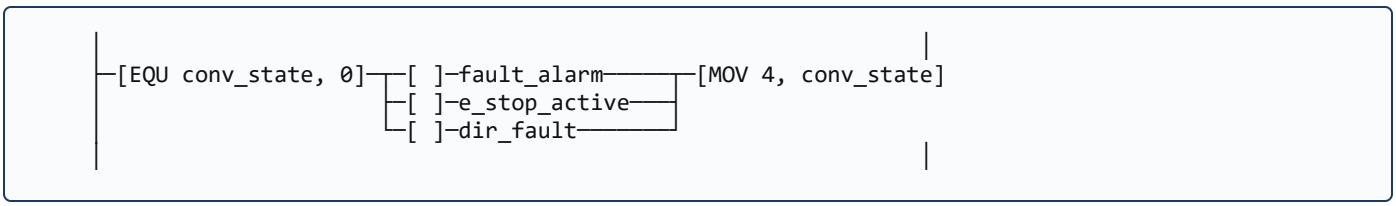
The conveyor has 5 states: `0 = IDLE`, `1 = STARTING`, `2 = RUNNING`, `3 = STOPPING`, `4 = FAULT`. The state variable is `conv_state : INT`. Each transition is a rung that, when its conditions are met, uses `[MOV new_state, conv_state]` to change states.

Heads-up: ladder state machines get verbose fast
The deployed ST version is one `CASE conv_state OF` block with ~80 lines. The ladder version is ~20 rungs that all reference the state via EQU compare contacts. Both are correct; the ladder version is more

> visual and easier to step through with a finger on a printout, but every new state means N more transition rungs. Five states is the sweet spot; ten states starts to feel painful.

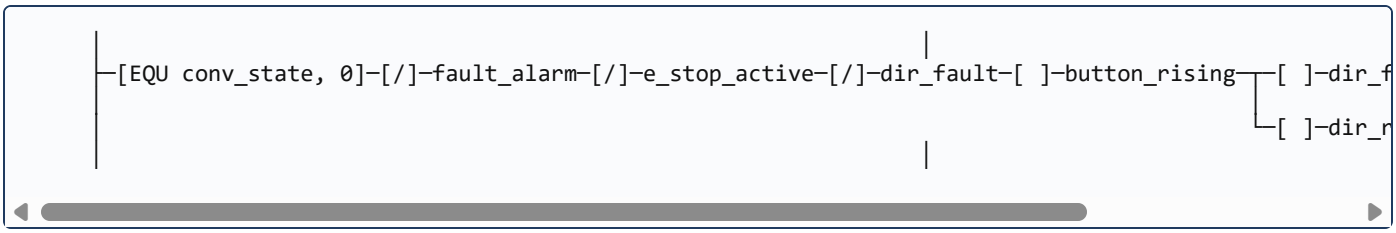
Transitions out of state 0 (IDLE)

Rung 8-1 — IDLE → FAULT on any fault condition



Three parallel branches OR'd together — any one TRUE means transition to state 4.

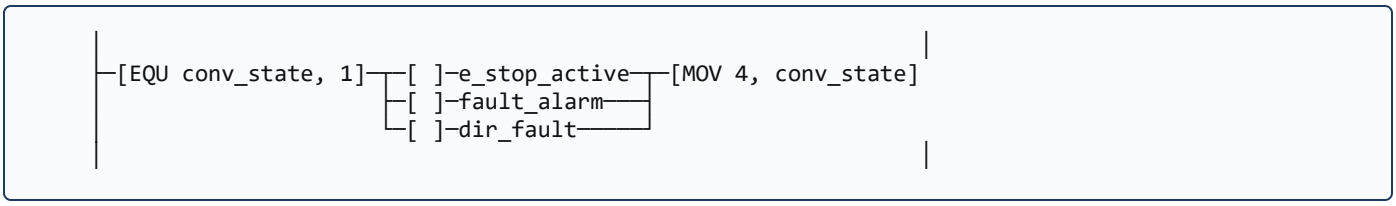
Rung 8-2 — IDLE → STARTING on button + direction selected + no fault



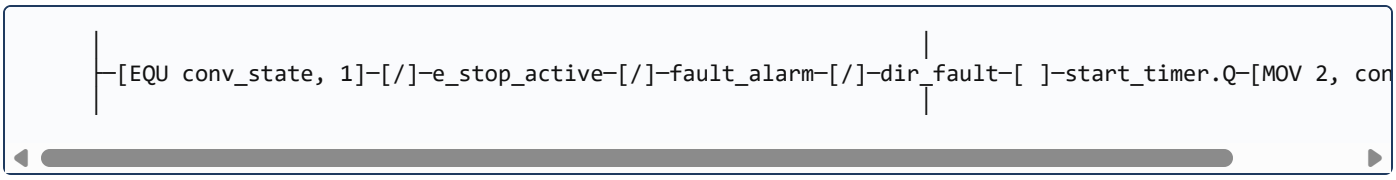
All conditions must be true. Direction selector parallel: FWD OR REV (but not OFF).

Transitions out of state 1 (STARTING)

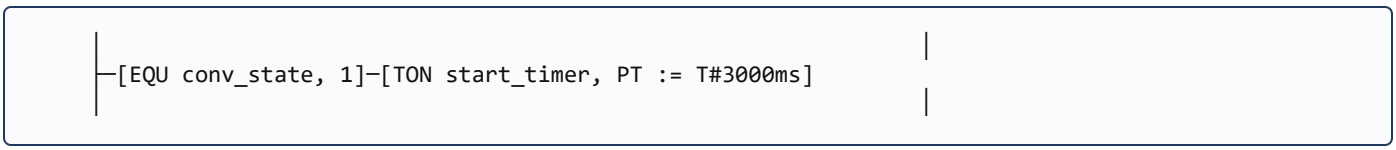
Rung 8-3 — STARTING → FAULT



Rung 8-4 — STARTING → RUNNING on start_timer.Q



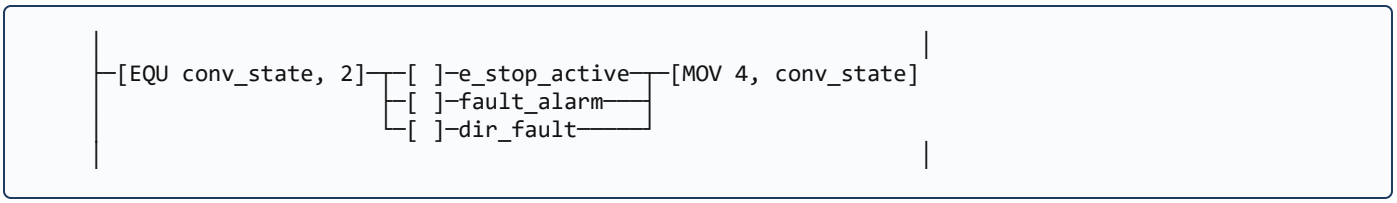
Rung 8-5 — Start timer drives



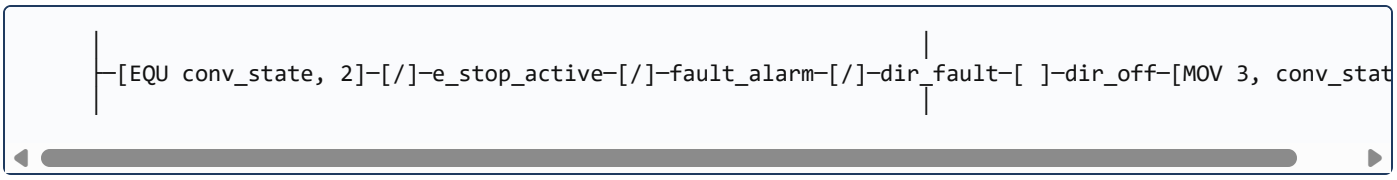
TON is called every scan; IN tied to "are we in state 1?". When IN drops (state moves on), TON resets automatically.

Transitions out of state 2 (RUNNING)

Rung 8-6 — RUNNING → FAULT

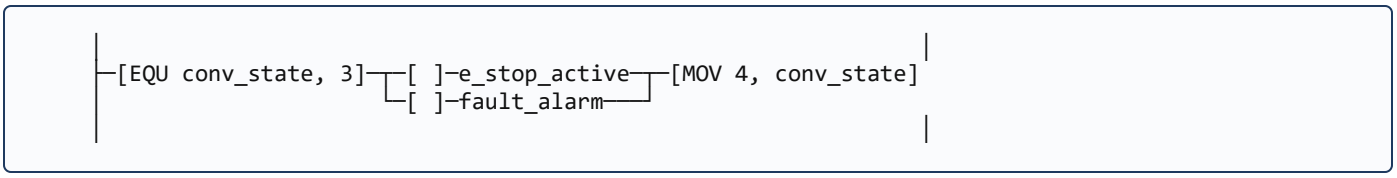


Rung 8-7 — RUNNING → STOPPING on dir_off (normal stop)

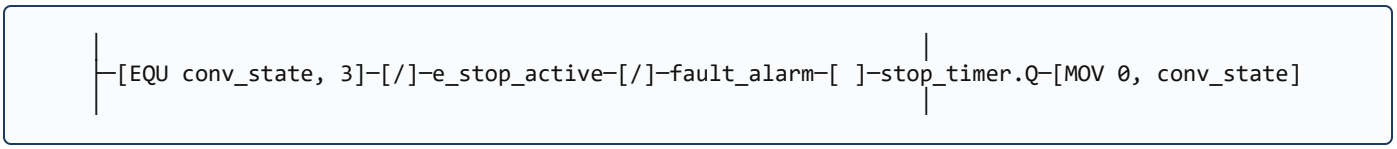


Transitions out of state 3 (STOPPING)

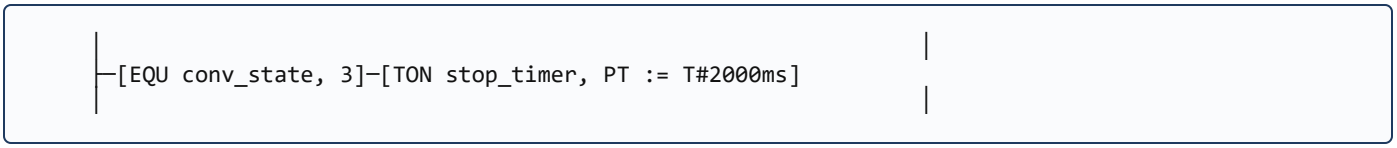
Rung 8-8 — STOPPING → FAULT



Rung 8-9 — STOPPING → IDLE on stop_timer.Q

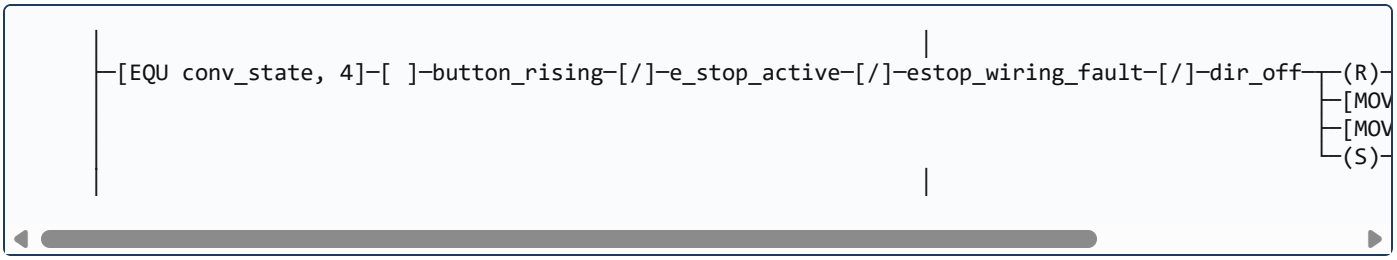


Rung 8-10 — Stop timer drives



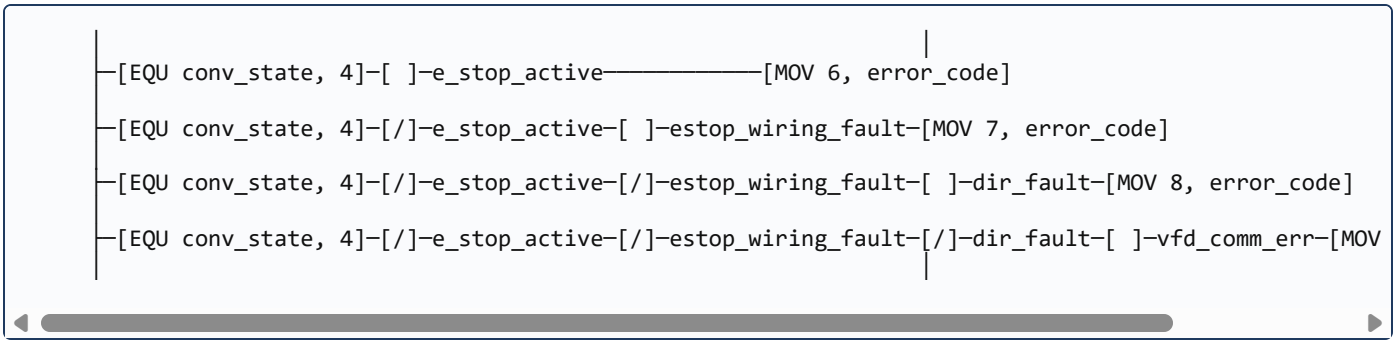
Transitions out of state 4 (FAULT)

Rung 8-11 — FAULT → IDLE on reset (PB + e-stop clear + selector not OFF)



Error-code tagging while in FAULT

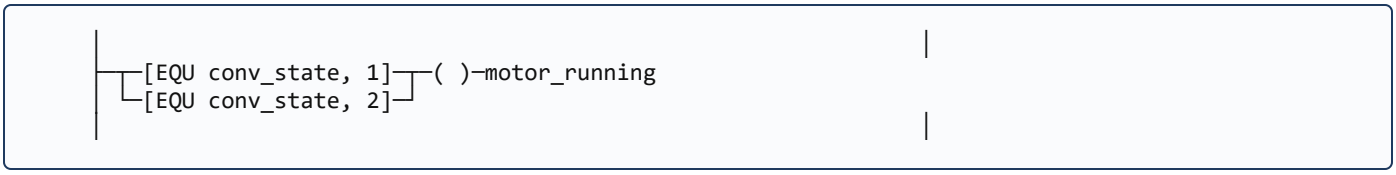
Rung 8-12 — Identify fault source (lower-priority faults at bottom)



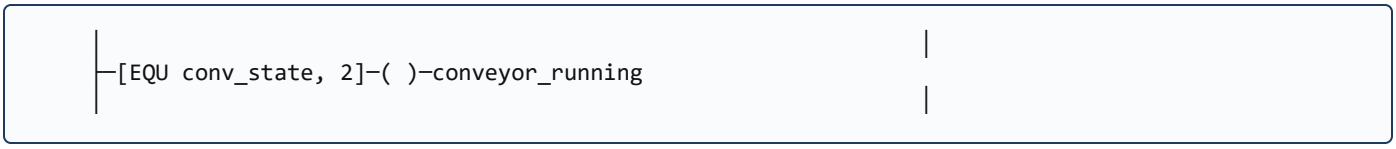
XIO chains to make the priority explicit — first-true-wins.

9 motor_running and conveyor_running bits — 4 rungs

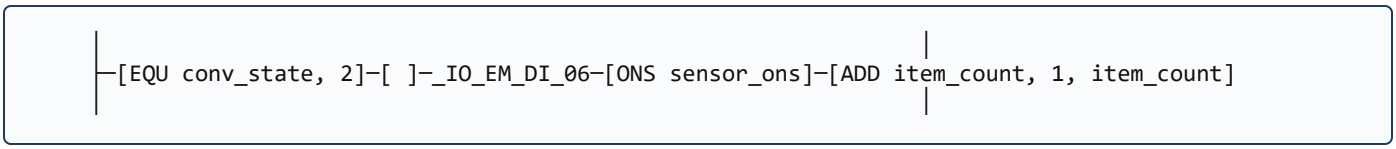
Rung 9-1 — motor_running while in STARTING or RUNNING



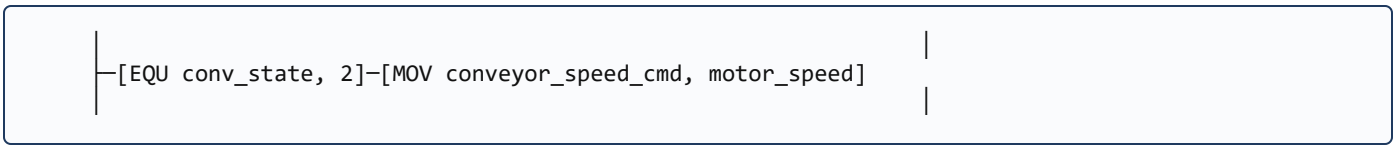
Rung 9-2 — conveyor_running in RUNNING only



Rung 9-3 — Item count on rising-edge of exit sensor (RUNNING only)

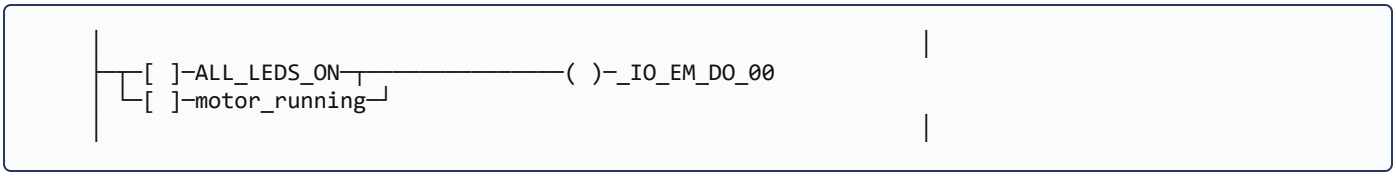


Rung 9-4 — Speed setpoint pass-through (RUNNING only)

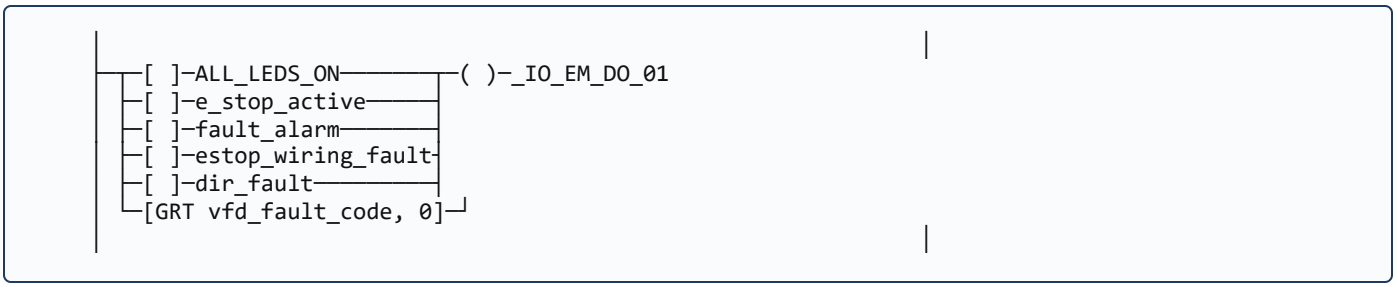


10 LED indicators — 3 rungs

Rung 10-1 — Green pilot (running)



Rung 10-2 — Red pilot (fault / e-stop)



Rung 10-3 — Pushbutton lamp (mirrors motor_running)

PART 3 — Prog3-Modbus-LD (all in ladder)

11 Config struct loading — one rung per field

The four `MSG_MODBUS` instances each need a `LocalCfg` struct (channel, function code, element count) and a `TargetCfg` struct (register address, slave node). In ST these load with simple assignments; in ladder each field is one MOV rung.

Why this matters: CCW lets you tag a rung as "First Scan only" or you can wire to an "always TRUE" bit, but most programmers just leave the MOV rungs unconditioned and let them load every scan. The performance cost is negligible (~1 µs per MOV) and it means a runtime corruption can't leave stale config values in a struct.

Read monitor cfg — 5 rungs

```
│                                                              │
├─[MOV 2,    read_local_cfg.Channel]
├─[MOV 0,    read_local_cfg.TriggerType]
├─[MOV 3,    read_local_cfg.Cmd]              (* FC03 = read holding *)
├─[MOV 4,    read_local_cfg.ElementCnt]       (* 4 regs: 0x2103..0x2106 *)
├─[MOV 8451, read_target_cfg.Addr]            (* 0x2103 *)
├─[MOV 1,    read_target_cfg.Node]
│                                                              │
```

Write command cfg — 5 rungs

```
│                                                              │
├─[MOV 2,    write_cmd_local_cfg.Channel]
├─[MOV 0,    write_cmd_local_cfg.TriggerType]
├─[MOV 6,    write_cmd_local_cfg.Cmd]         (* FC06 = write single *)
├─[MOV 1,    write_cmd_local_cfg.ElementCnt]
├─[MOV 8192, write_cmd_target_cfg.Addr]       (* 0x2000 control *)
├─[MOV 1,    write_cmd_target_cfg.Node]
│                                                              │
```

Write frequency cfg — 5 rungs

```
│                                                              │
├─[MOV 2,    write_freq_local_cfg.Channel]
├─[MOV 0,    write_freq_local_cfg.TriggerType]
├─[MOV 6,    write_freq_local_cfg.Cmd]
├─[MOV 1,    write_freq_local_cfg.ElementCnt]
├─[MOV 8193, write_freq_target_cfg.Addr]      (* 0x2001 freq setpoint *)
├─[MOV 1,    write_freq_target_cfg.Node]
│                                                              │
```

Fault reset cfg — 5 rungs

```
|                                                                    |
|--[MOV 2,    fault_reset_local_cfg.Channel]
|--[MOV 0,    fault_reset_local_cfg.TriggerType]
|--[MOV 6,    fault_reset_local_cfg.Cmd]
|--[MOV 1,    fault_reset_local_cfg.ElementCnt]
|--[MOV 8194, fault_reset_target_cfg.Addr]     (* 0x2002 fault reset *)
|--[MOV 1,    fault_reset_target_cfg.Node]
|                                                                    |
```

> **20 MOV rungs to do what 20 ST lines do**
> ST version: 20 assignments in 20 lines. LD version: 20 rungs, each with one MOV instruction. The visual chrome takes more space but you can put a finger on a printout and verify each value against the cheatsheet. This is exactly where ST starts to feel attractive — see Part 6 §21.

12 COP write-buffer loading — 3 rungs

Each FC06 write needs its payload (one register) in an array slot the MSG block reads from. Use COP to copy the source variable into `*_data(1)`.

```
|                                                                    |
|--[COP vfd_cmd_word,      write_cmd_data,    1]   (* command word *)
|--[COP vfd_freq_setpoint, write_freq_data,   1]   (* freq × 10 *)
|--[COP fault_reset_cmd,   fault_reset_data,  1]   (* always 2 *)
|                                                                    |
```

Also load the constants once

```
|                                                                    |
|--[MOV 2, fault_reset_cmd]                 (* always 2 to clear faults *)
|                                                                    |
```

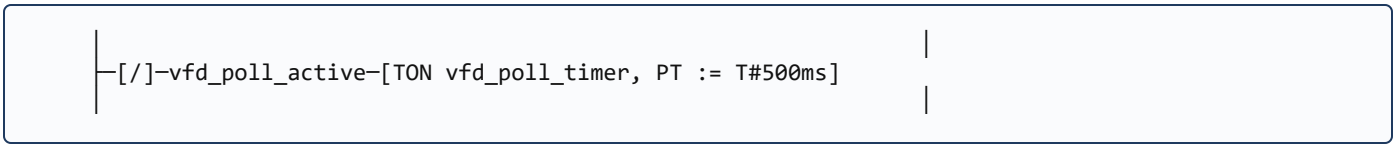
Frequency setpoint computation

```
|                                                                    |
|--[ ]--motor_running--[GRT conveyor_speed_cmd, 0]--[MUL conveyor_speed_cmd, 10, vfd_freq_setpoint]
|
|--[ ]--motor_running--[EQU conveyor_speed_cmd, 0]--[MOV 300, vfd_freq_setpoint]
|                                                                    |
```

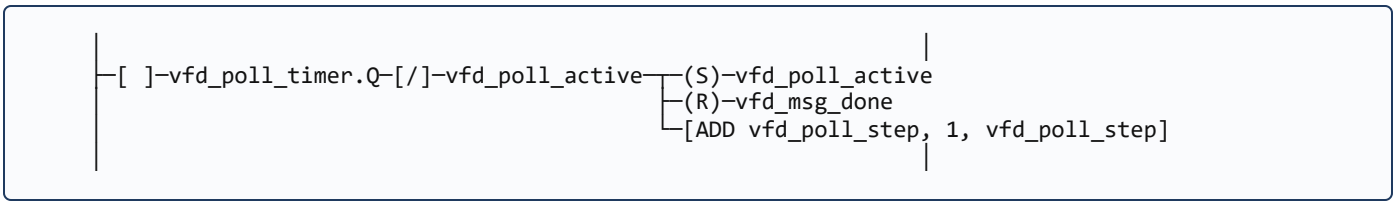
If a non-zero speed is commanded, scale Hz×10. If zero while motor_running (operator forgot to set speed), default to 30.0 Hz so the motor doesn't sit at 0 Hz dancing.

13 Polling timer + step advance — 3 rungs

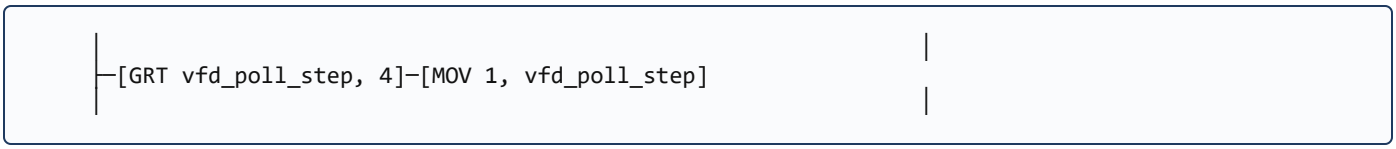
Rung 13-1 — Poll timer ticks while not actively polling



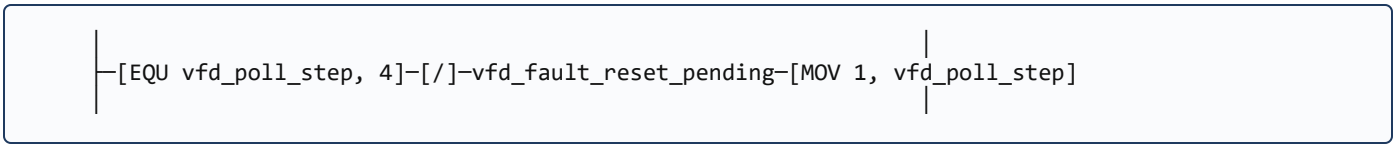
Rung 13-2 — On timer.Q: arm next poll, increment step



Rung 13-3 — Wrap step 5+ back to 1

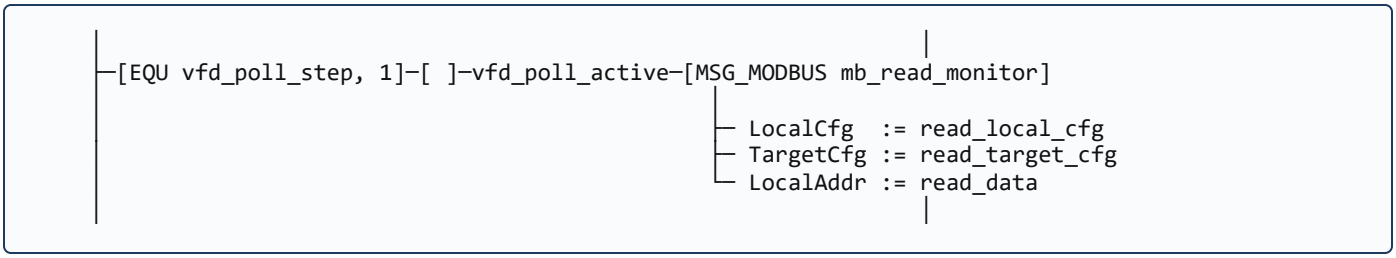


Rung 13-4 — Skip fault-reset step (step 4) unless a reset is pending

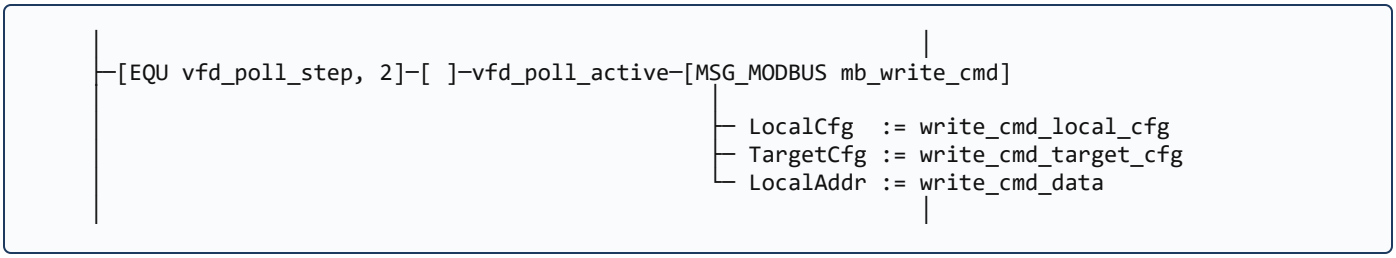


14 The 4 polling MSG rungs

Rung 14-1 — Read monitor block (step 1)



Rung 14-2 — Write command (step 2)



Rung 14-3 — Write frequency (step 3)

```
|                                                                    |
|-[EQU vfd_poll_step, 3]-[ ]-vfd_poll_active-[MSG_MODBUS mb_write_freq]
|                                                  |
|                                                  |- LocalCfg  := write_freq_local_cfg
|                                                  |- TargetCfg := write_freq_target_cfg
|                                                  |- LocalAddr := write_freq_data
|                                                                    |
```

Rung 14-4 — Fault reset (step 4)

```
|                                                                    |
|-[EQU vfd_poll_step, 4]-[ ]-vfd_poll_active-[MSG_MODBUS mb_fault_reset]
|                                                  |
|                                                  |- LocalCfg  := fault_reset_local_cfg
|                                                  |- TargetCfg := fault_reset_target_cfg
|                                                  |- LocalAddr := fault_reset_data
|                                                                    |
```

15 Check MSG results — per-step result rungs

Rung 15-1 — Step 1 read success

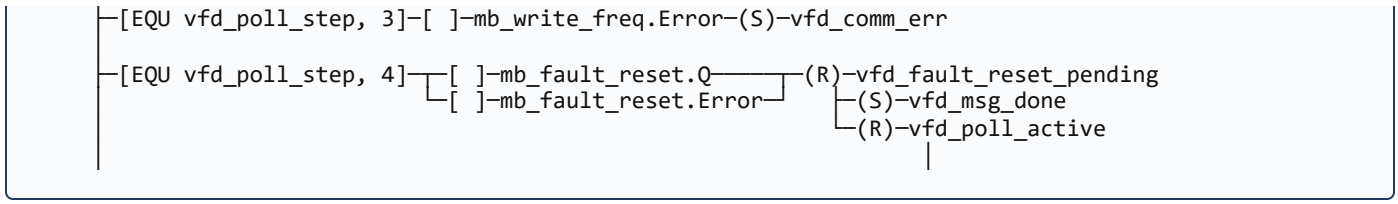
```
|                                                                    |
|-[EQU vfd_poll_step, 1]-[ ]-mb_read_monitor.Q--|-[COP read_data, vfd_frequency, 1]   (* Hz × 10 *)
|                                               |-[COP read_data[1], vfd_current,  1]
|                                               |-[COP read_data[2], vfd_dc_bus,   1]
|                                               |-[COP read_data[3], vfd_voltage,  1]
|                                               |-(S)-vfd_comm_ok
|                                               |-(R)-vfd_comm_err
|                                               |-(S)-vfd_msg_done
|                                               |-(R)-vfd_poll_active
|                                                                    |
```

Rung 15-2 — Step 1 read error

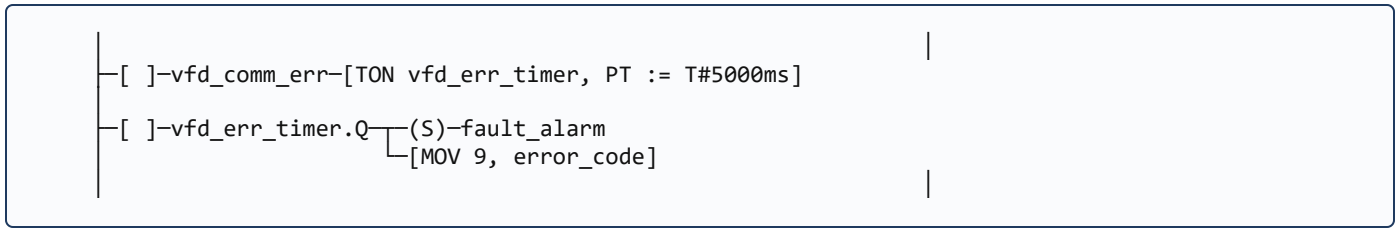
```
|                                                                    |
|-[EQU vfd_poll_step, 1]-[ ]-mb_read_monitor.Error--|-(R)-vfd_comm_ok
|                                                   |-(S)-vfd_comm_err
|                                                   |-(S)-vfd_msg_done
|                                                   |-(R)-vfd_poll_active
|                                                                    |
```

Rung 15-3..5 — Steps 2/3/4 done (same pattern, condensed)

```
|                                                                    |
|-[EQU vfd_poll_step, 2]--|-[ ]-mb_write_cmd.Q------|-(S)-vfd_msg_done
|                         |-[ ]-mb_write_cmd.Error-|   |-(R)-vfd_poll_active
|
|-[EQU vfd_poll_step, 2]-[ ]-mb_write_cmd.Error-(S)-vfd_comm_err
|
|-[EQU vfd_poll_step, 3]--|-[ ]-mb_write_freq.Q------|-(S)-vfd_msg_done
|                         |-[ ]-mb_write_freq.Error-|   |-(R)-vfd_poll_active
|
```



Rung 15-6 — Watchdog: latch fault_alarm if comm error persists 5s



PART 4 — motor_starter as an LD UDFB

16 Create the UDFB in ladder language

CCW supports User-Defined Function Blocks written in any IEC 61131-3 language — Ladder Diagram included. Same external interface as the ST version (same pins, same instance variables), just the body is drawn as rungs instead of typed as ST.

Step 1 — Create the LD UDFB

1. ☐ Project Organizer → right-click **User-Defined Function Blocks** → **Add** → **New Function Block**.
2. ☐ Language: **Ladder Diagram**. (Not Structured Text — that's the other doc.)
3. ☐ Name: `motor_starter_LD`. The `_LD` suffix prevents collision with the ST version if both projects ever merge.
4. ☐ Empty variable list + empty rung area opens.

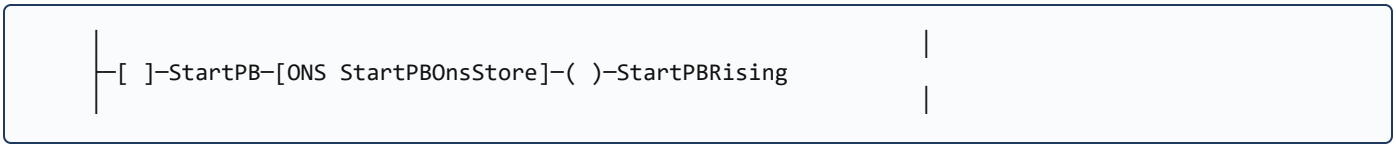
Step 2 — Declare variables (same as the ST version)

Section	Name	Type	Initial
VAR_INPUT	`Enable`	`BOOL`	`TRUE`
VAR_INPUT	`StartPB`	`BOOL`	`FALSE`
VAR_INPUT	`DirFwdSw`	`BOOL`	`FALSE`
VAR_INPUT	`DirRevSw`	`BOOL`	`FALSE`
VAR_INPUT	`EStopOK`	`BOOL`	`FALSE`
VAR_OUTPUT	`VFDCmdWord`	`INT`	`1`
VAR_OUTPUT	`RunLamp`	`BOOL`	`FALSE`
VAR	`RunLatch`	`BOOL`	`FALSE`
VAR	`DirLatchFwd`	`BOOL`	`FALSE`
VAR	`DirLatchRev`	`BOOL`	`FALSE`
VAR	`StartPBOnsStore`	`BOOL`	`FALSE`
VAR	`StartPBRising`	`BOOL`	`FALSE`
VAR	`RunOK`	`BOOL`	`FALSE`

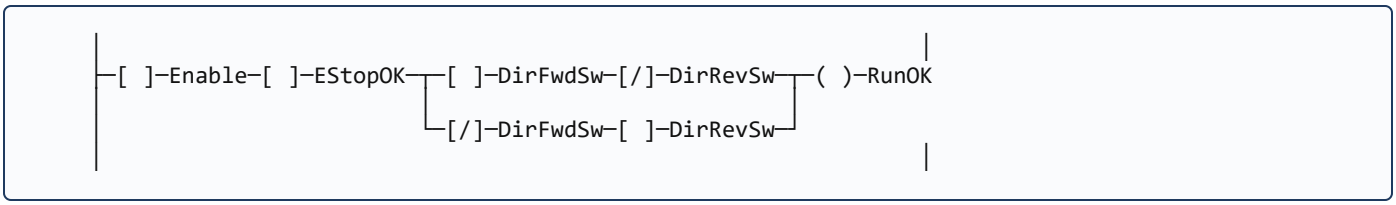
Why this matters: Notable change from the ST version: `DirLatch : INT` (0/1/2) becomes two BOOLs `DirLatchFwd` + `DirLatchRev`. Ladder works better with bool latches than with integer compares for state this small. ST's integer is more compact in text; ladder's two bools are easier to draw and easier to debug visually.

17 The 5 UDFB body rungs

Rung 17-1 — Rising edge on StartPB

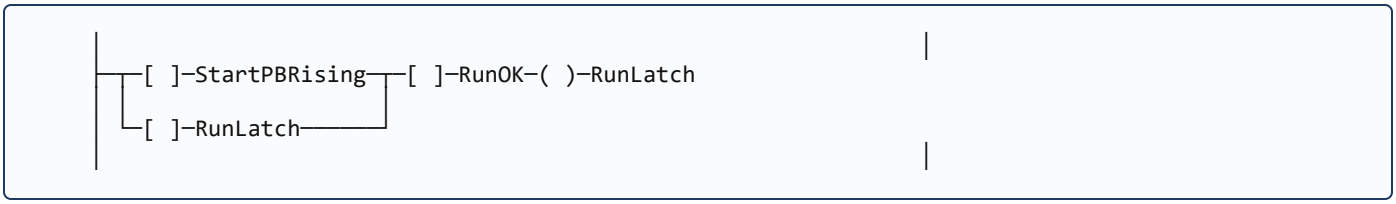


Rung 17-2 — RunOK composite (master enable + safety + direction selected)



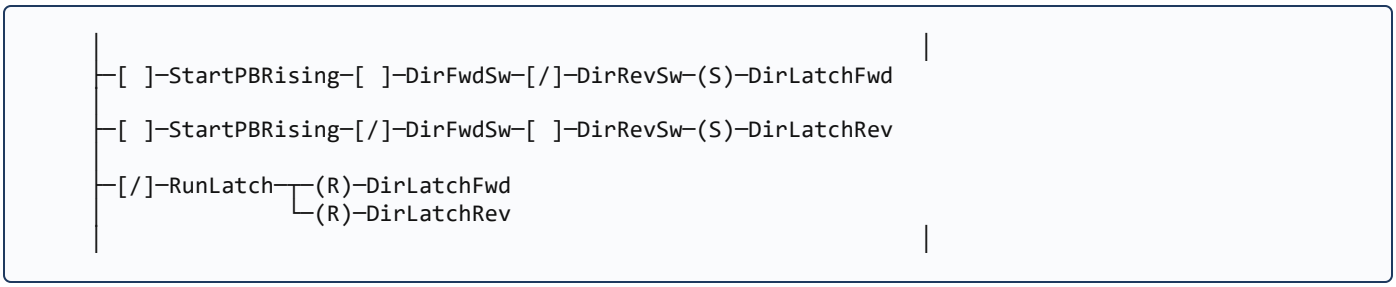
Parallel branch: FWD-only OR REV-only. Both-true (wiring fault) and neither-true (OFF) both produce RunOK = FALSE.

Rung 17-3 — Seal-in latch on RunLatch



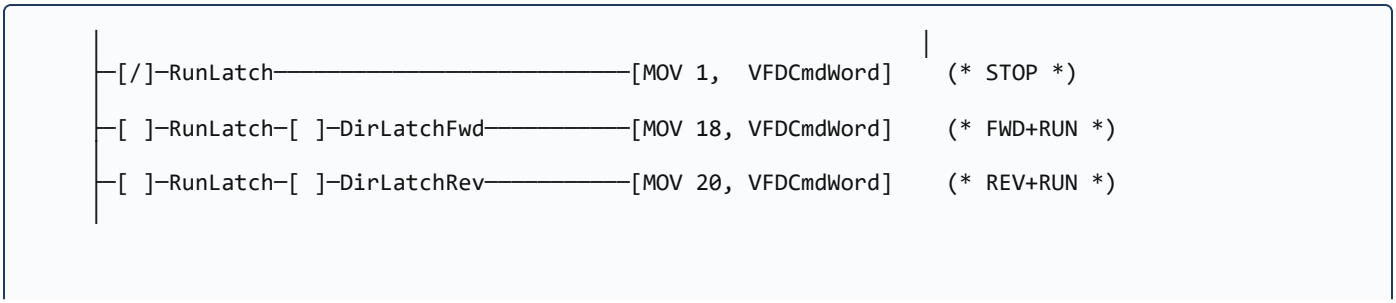
Classic seal-in. Once RunLatch sets, it holds itself through the lower branch as long as RunOK stays TRUE. The moment RunOK drops (any drop condition), RunLatch coils OFF.

Rung 17-4 — Capture direction at the moment RunLatch sets



Set the appropriate direction latch on the edge; clear both whenever RunLatch drops.

Rung 17-5 — Output assignment: VFDCmdWord + RunLamp



```
    ├─[ ]─RunLatch─( )─RunLamp
    │                                                          │
```

> **Drop-on-direction-change handled by Rung 17-2 alone**
> In the ST version, "drop on direction change while running" was a named local (`DirChangedWhileRunning`) computed before the latch IF. In ladder, it's implicit: if the operator flips the selector mid-run, RunOK's parallel branch fails (e.g., DirLatchFwd is still TRUE but DirFwdSw is now FALSE → neither parallel branch passes → RunOK drops → seal-in coils OFF). One rung, no extra named bool needed.

Step 3 — Build the UDFB

1. ☐ Press **F8**. Look for **0 errors**.
2. ☐ Errors? Typo in a variable name on a rung vs the variable list. CCW is spelling-sensitive even though it's case-insensitive.

18 Call the UDFB from Prog4-Motor-LD

Rung 18-1 — One instance, all pins wired

```
    │                                                          │
    ├─[ ]─enable_motor_ctrl─[motor_starter_LD  motor_starter_inst]
    │                              │
    │                              ├─ Enable     := enable_motor_ctrl
    │                              ├─ StartPB    := _IO_EM_DI_04
    │                              ├─ DirFwdSw   := dir_fwd
    │                              ├─ DirRevSw   := dir_rev
    │                              ├─ EStopOK    := e_stop_ok
    │                              ├─ VFDCmdWord => vfd_cmd_word
    │                              └─ RunLamp    => _IO_EM_DO_00
    │                                                          │
```

Why this matters: Dragging the UDFB onto the rung gives you a box with one pin per input/output. `:=` for inputs, `=>` for outputs. CCW also lets you skip the leading enable contact if Enable defaults to TRUE — but wiring an explicit enable bit makes it easy to globally pause motor control without touching the field switches.

19 Important — comment out the old state-machine writes to vfd_cmd_word

If you ported the deployed Prog2 ST state machine to Prog2-Safety LD per Section 8, then Section 8 doesn't actually write `vfd_cmd_word` anywhere — clean. Skip this step.

If instead you're running the deployed Prog2 ST *and* this new Prog4-Motor-LD side-by-side in the same project (mixed-mode), you have to comment out the same six sites from the ST version that the Complete doc §20 Fix 5 lists. Two writers in one scan = last-writer-wins = random behavior.

PART 5 — Test + troubleshoot

20 Test procedure (LD edition)

1. ☐ Build (F8). 0 errors. Unknown-variable errors mean a global is missing from the Controller Variables table — declare it and rebuild.
2. ☐ Download via USB.
3. ☐ PLC to Run.
4. ☐ Online → Variable Monitor. Pin `conv_state`, `vfd_cmd_word`, `vfd_poll_step`, `motor_starter_inst.RunLatch`.

Test 1 — E-stop XOR proves all four quadrants

✓**You should see:** Released → `e_stop_active` = FALSE, `estop_wiring_fault` = FALSE. Pressed → `e_stop_active` = TRUE, `estop_wiring_fault` = FALSE. Wire-cut one channel (simulate by forcing one of the inputs) → both bits latch TRUE, `fault_alarm` latches TRUE, requires reset.

Test 2 — State machine cycles IDLE → STARTING → RUNNING → STOPPING → IDLE

✓**You should see:** Selector FWD, press PB. `conv_state`: 0 → 1 (for 3 sec while `start_timer` runs) → 2 (running). Flip selector to OFF. `conv_state`: 2 → 3 (for 2 sec while `stop_timer` runs) → 0.

Test 3 — State machine catches every fault

✓**You should see:** Press e-stop from any non-FAULT state. `conv_state` → 4 within one scan. `error_code` = 6 (e-stop). Press PB while e-stop is released + selector not OFF → `conv_state` → 0, `error_code` → 0, `fault_alarm` → FALSE.

Test 4 — Modbus sequencer rotates 1→4 (or 1→3 when no reset pending)

✓**You should see:** `vfd_poll_step` visibly cycles. `mb_read_monitor.ErrorID` = 0 sustained. `read_data(1)` > 0 when motor running.

Test 5 — UDFB seal-in works

✓**You should see:** Selector FWD, press PB. `motor_starter_inst.RunLatch` goes TRUE for one scan and stays TRUE. `motor_starter_inst.DirLatchFwd` = TRUE. `vfd_cmd_word` = 18. Flip selector to REV mid-run — RunLatch drops within one scan, motor stops, must re-press PB.

Test 6 — E-stop drops UDFB latch

✓**You should see:** Motor running, press e-stop. `e_stop_ok` → FALSE → UDFB's `EStopOK` pin → FALSE → RunOK FALSE → RunLatch drops. `vfd_cmd_word` = 1.

Test 7 — E-stop release alone does NOT auto-restart

✓**You should see:** Release e-stop. RunLatch stays FALSE because StartPBRising is FALSE (you haven't pressed PB again). Operator must press PB to restart.

21 Troubleshooting — ladder-specific gotchas

Symptom	Likely cause	Fix
State machine "stuck" in one state forever	Transition rung for that state is wrong; or the condition contact's underlying global isn't being driven	Variable monitor: watch the transition rung's conditions one by one. Whichever XIC/XIO is the wrong color (red = TRUE in CCW online) is the contact blocking progress.
Two state-transition rungs fire on the same scan	Rung order matters — earlier rungs win in CCW. If rung 8-1 (IDLE→FAULT) and rung 8-2 (IDLE→STARTING) both true (e.g., button pressed at the moment a fault appears), 8-1 wins because it executes first.	Order from highest priority (faults) to lowest (normal transitions) within each state. The existing ordering is correct — keep it.
Edge bit (button_rising) stays TRUE for multiple scans	Using OTE pattern (rung 6-1 alternate) but the two rungs are in the wrong order. Edge rung must run BEFORE prev-shift rung.	Reorder the rungs in Prog2-Safety. ONS instruction (rung 6-1 preferred) avoids this entirely.
MOV rung loads stale value	You wired the MOV source to a tag that isn't being driven this scan	Verify the source variable updates somewhere upstream. `conveyor_speed_cmd` for instance — does anything assign it? If no, it sits at zero.
UDFB instance pin shows "no variable" in monitor	You opened the UDFB **definition**, not the instance	Project Organizer → Prog4-Motor-LD → expand → `motor_starter_inst` in locals → monitor it.
RunLatch stays TRUE on first scan after download even though motor wasn't running before	CCW retained instance memory across download	Project → Clear Memory Retention → re-download. Or power-cycle PLC.
Build error: "unknown identifier `ONS_storage_bit`"	ONS instruction needs its storage bit declared as a global	Global Variables → declare `pb_ons_storage : BOOL` (or whatever name the rung uses). Rebuild.
Modbus rungs fire but ErrorID = 255 on every one	Channel := 0 instead of 2	Verify Rung 11-* MOVs all load value **2** into the .Channel field. Re-download.
Two rungs both drive the same output coil (e.g., `_IO_EM_DO_00`)	CCW silently OR's them. Sometimes intentional; usually a mistake. Last-rung-wins is NOT how it works — both rungs are evaluated and the output reflects the OR.	Audit Prog2-Safety §10 rungs. The green pilot should only be driven from one place (Rung 10-1).

PART 6 — When ladder hurts (and where ST starts to look attractive)

22 Honest map of where each language wins

Concern	Ladder	ST	Verdict
E-stop XOR supervision	4 rungs, very visual	1 IF, 6 lines	Ladder if the electrician will be the one tracing it. ST if it's pure code review.
I/O decoding (dir_fwd / dir_rev / dir_off / dir_fault)	4 rungs, dead simple	4 assignments, even simpler	Toss-up. Use the project's primary language.
Edge detection on a pushbutton	1 ONS rung	2 lines (current + prev)	Ladder is more idiomatic. Both fine.
5-state state machine	**~20 transition rungs + 5 sub-state output rungs**	**One CASE block, ~30 lines**	**ST wins decisively at 5+ states. Ladder still fine at 2–3 states.**
Modbus config struct loading (20+ fields)	20+ MOV rungs	20+ assignments, half the page space	ST wins on space, but ladder lets a tech verify each value against a cheatsheet without scrolling.
COP buffer loading (3 sites)	3 COP rungs	3 cop_*(...) calls	Toss-up.
Polling timer + step advance	3 rungs	4 lines + 1 IF	Toss-up.
MSG_MODBUS calls (4 sites)	4 rungs, very clear	4 FB calls, also clear	Ladder slightly clearer for first-time readers.
MSG result handling (4 steps × ok/err)	~10 rungs total	One CASE block, ~30 lines	ST is more compact. Ladder is more visual. Either works.
UDFB body (motor_starter)	5 rungs	~30 lines of ST	Ladder wins on visual clarity for this size. ST wins if the UDFB grows past ~10 internal states.
LED indicator outputs	3 rungs with OR branches	3 IF lines	Toss-up.
Error-code tagging in FAULT state	4 priority rungs with XIO chains	4 ELSIF lines, much tighter	ST wins on conciseness; ladder wins on visual priority chain.

The honest summary

Ladder wins when: a 3rd party (electrician, maintenance tech, safety auditor) will read the program; the logic is parallel and short; you want a printout that can be verified contact-by-contact against the field wiring.

ST wins when: state machines have more than ~3 states; you're doing arithmetic, scaling, or formatted output; the function has 20+ variables interacting and ladder's spatial layout would force horizontal scrolling.

Most production programs are mixed. The deployed `Micro820_v4.1.8_Program.st` uses ST for the state machine (where it shines) and the same project has ladder for the Modbus polling rungs (where the MSG block calls read most naturally). That's not a code smell; it's IEC 61131-3 design intent. The deployed program leans ST-heavy because it grew organically from a CCW template; a from-scratch design today might lean more ladder for the safety/I/O sections and keep ST only for the state machine + Modbus result CASE.

When you finish this doc — what to read next

1. **`Conv_Simple_Complete.pdf`** — read the ST version of the same logic. You'll see exactly which ladder rungs collapse into one ST line and where the trade-offs sit.
2. **`plc/Micro820_v4.1.8_Program.st`** — the deployed program. By the time you can read both languages, this file should look familiar to you — every section maps to a Part in this doc.
3. **`plc/LD_BUILD_GUIDE_v5.md`** — the older partial ladder reference. Superseded by this doc (which corrects the Channel = 0 / ElementCnt = 6 / mb_read_status bugs), but historically useful.

🛈 Tear-off appendix — ladder edition

Three programs, in execution order

1. **Prog2-Safety** — e-stop XOR (4 rungs), dir decode (4), edge detect (1–2), contactor (1), state machine (~20), motor flags (4), LEDs (3) = ~37 rungs
2. **Prog3-Modbus-LD** — cfg loading (20), COP buffers (3), freq scaling (2), poll timer (4), MSG calls (4), result handling (~10), watchdog (2) = ~45 rungs
3. **Prog4-Motor-LD** — UDFB instance call (1 rung); UDFB body internal (5 rungs)

Critical values (must be right)

All four .Channel MOVs load value	`2`
read_local_cfg.ElementCnt MOV loads	`4`
MSG instance name for monitor read	`mb_read_monitor` (NOT `mb_read_status`)
Frequency setpoint scaling	Hz × 10 (30 Hz → 300)
Cmd words (bench-verify)	STOP=1, FWD+RUN=18, REV+RUN=20

Ladder gotchas worth taping to the wall

- Rung execution order matters for state-machine transitions. Faults first, normals second.
- Two rungs to the same coil = OR, not last-wins. Audit your outputs.
- ONS storage bit MUST be a unique global per ONS instance. Sharing one means edge detection bleeds across rungs.
- UDFB instance vs definition: monitor the instance, not the definition.
- EQU/GRT compare contacts are FALSE when comparing to an uninitialized INT (typically 0). Initialize your state variable.